***BRACT's***

***VishwakarmaInstitute of Technology, Pune***

***PracticalImplementation Sheet***

Department:CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 8

Roll no: 44 PRN No: 12410016 Name of Student: ***Sahil Tagade***

***Subject Name : Deep Learning***

## *1. Aim*

To implement a pre-trained BERT model for sentiment analysis using a sample dataset, fine-tune the model for binary text classification, evaluate its performance, and classify new text reviews as positive or negative.

## *2. Objectives*

To understand the concept of pre-trained Transformer models.
To load and preprocess the IMDB movie review dataset.
To tokenize text using a BERT tokenizer.
To fine-tune a pre-trained BERT model for sentiment classification.
To evaluate the model using accuracy, precision, recall and F1-score.
To visualize model performance using training/validation loss and a confusion matrix.
To test the trained model on custom text reviews.

## *3. Introduction*

BERT stands for Bidirectional Encoder Representations from Transformers. It is a Transformer-based language model that is pre-trained on large amounts of text. BERT learns contextual representations of words by considering information from both directions of a sentence.

In this practical, the pre-trained bert-base-uncased model is fine-tuned for binary sentiment classification. The model receives movie reviews from the IMDB dataset and predicts whether a review is positive or negative.

### *3.1 Overall Workflow*

IMDBDataset
↓
Data Loading
↓
Data Sampling
↓
BERT Tokenization
↓

Pre-trained BERT
↓
Fine-Tuning
↓
Evaluation
↓
Confusion Matrix / Classification Report
↓
Custom Text Prediction

# *4. Software and Hardware Requirements*

## *Software Requirements:*

Google Colab
Python 3
PyTorch
Hugging Face Transformers
Hugging Face Datasets
Scikit-learn
Matplotlib
Seaborn

### *Hardware Requirement:*

A GPU such as Google Colab T4 is recommended for faster BERT fine-tuning. The implementation can also run on CPU, but training will generally take longer.

# *5. Dataset Description*

TheIMDB MovieReviewDatasetis used for sentiment analysis. Each review is associated with a binary sentiment label.

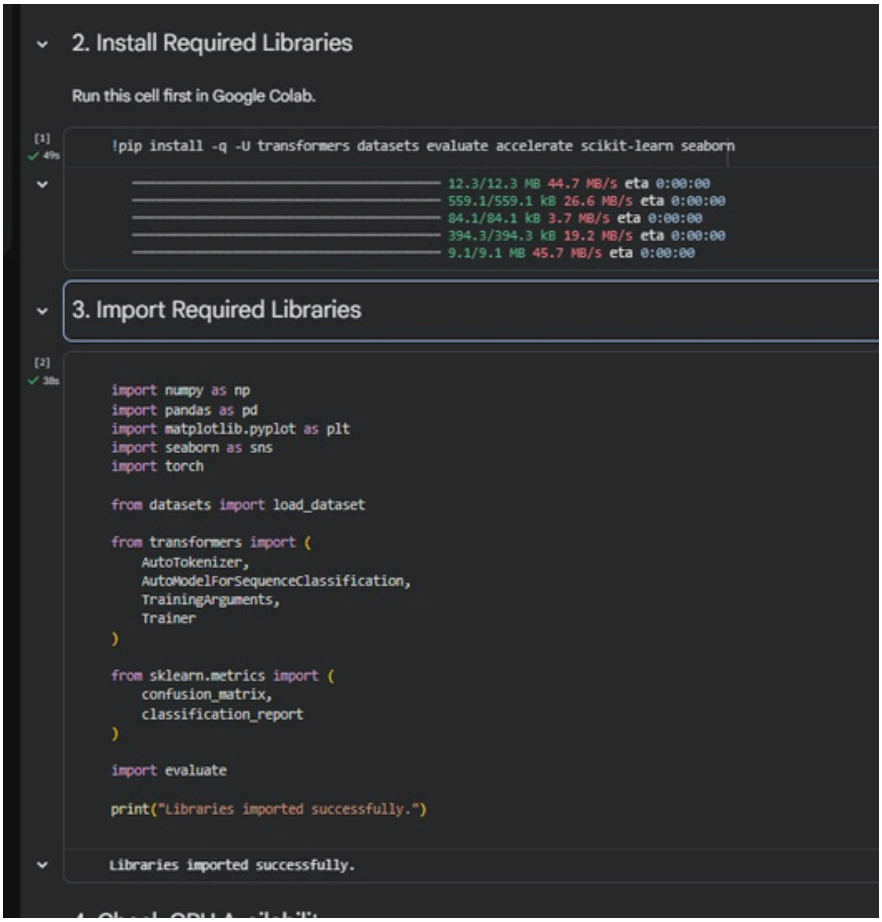
| Label | Meaning |
|---|---|
| 0 | Negative |
| 1 | Positive |

For this practical, a smaller subset is used to reduce execution time: 5,000 training reviews and 1,000 testing reviews.

# *6. Implementation*

## *6.1 Install Required Libraries*

```
!pip install -q -U transformers datasets evaluate accelerate scikit-learn seaborn
```




## *6.2 Import Required Libraries*

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import torch


from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)
from sklearn.metrics import confusion_matrix, classification_report
import evaluate
```

## *6.3 Check GPU Availability*

```
print("PyTorchversion:",torch.__version__)
print("GPU available:", torch.cuda.is_available())

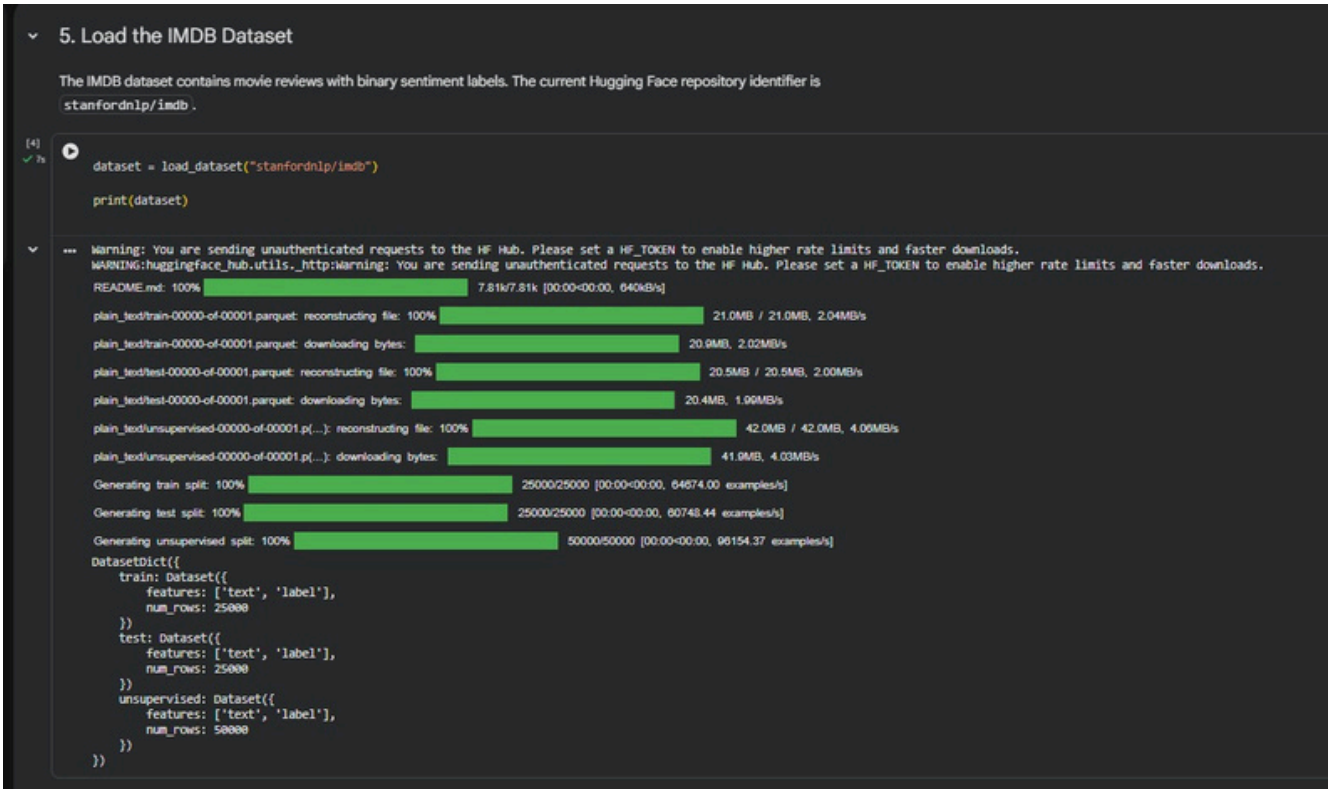
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
else:
    print("Running on CPU")
```




## *6.4 Load the IMDB Dataset*

dataset=load_dataset("stanfordnlp/imdb")
print(dataset)

The dataset isdownloaded automatically from Hugging Face. The repository identifier stanfordnlp/imdb is used for the currentdataset-loading interface.




## *6.5 Inspect a Sample Review*

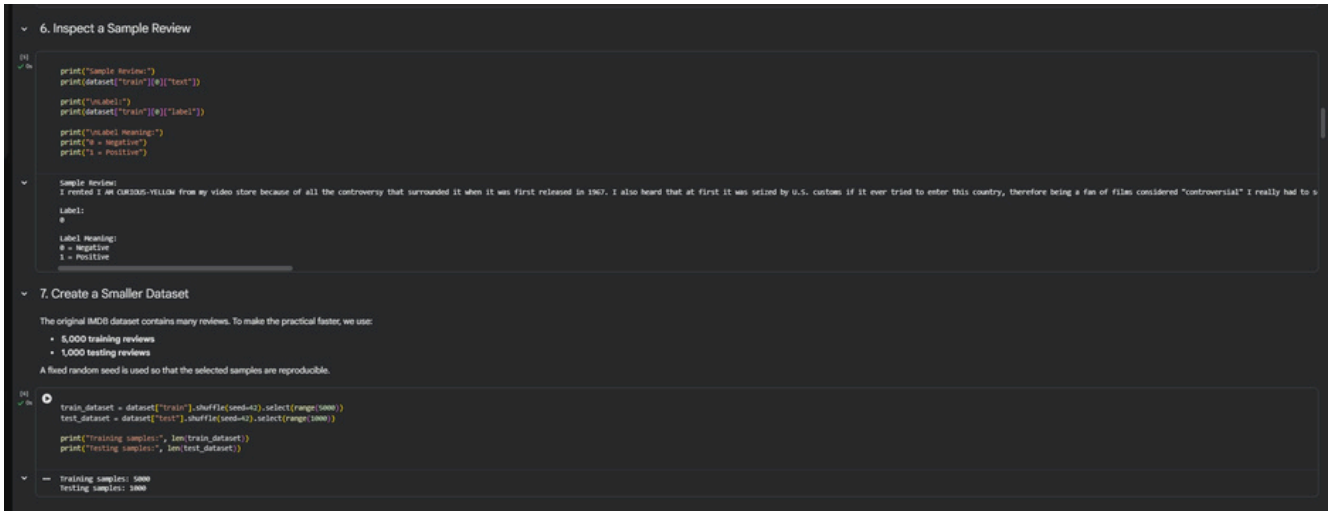
print("Sample Review:")
print(dataset["train"][0]["text"])

print("\nLabel:")
print(dataset["train"][0]["label"])

### *6.6 Create Training and Testing Subsets*

```
train_dataset = dataset["train"].shuffle(seed=42).select(range(5000))
test_dataset = dataset["test"].shuffle(seed=42).select(range(1000))

print("Training samples:", len(train_dataset))
print("Testing samples:", len(test_dataset))
```




# *7. BERT Tokenization*

Atokenizerconverts raw text intonumerical representations that can be processed by BERT. The maximum sequence length is set to 128 tokens. Longer reviews are truncated and shorter reviews are padded.

```
model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

Tokenization demonstration:

```
text = "I loved this movie."
tokens = tokenizer.tokenize(text)

print("Original text:")
print(text)

print("\nTokens:")
print(tokens)

print("\nToken IDs:")
print(tokenizer.convert_tokens_to_ids(tokens))
```




## *7.1 Tokenize the Dataset*

```
def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

train_tokenized = train_dataset.map(tokenize_function, batched=True)
test_tokenized = test_dataset.map(tokenize_function, batched=True)
```

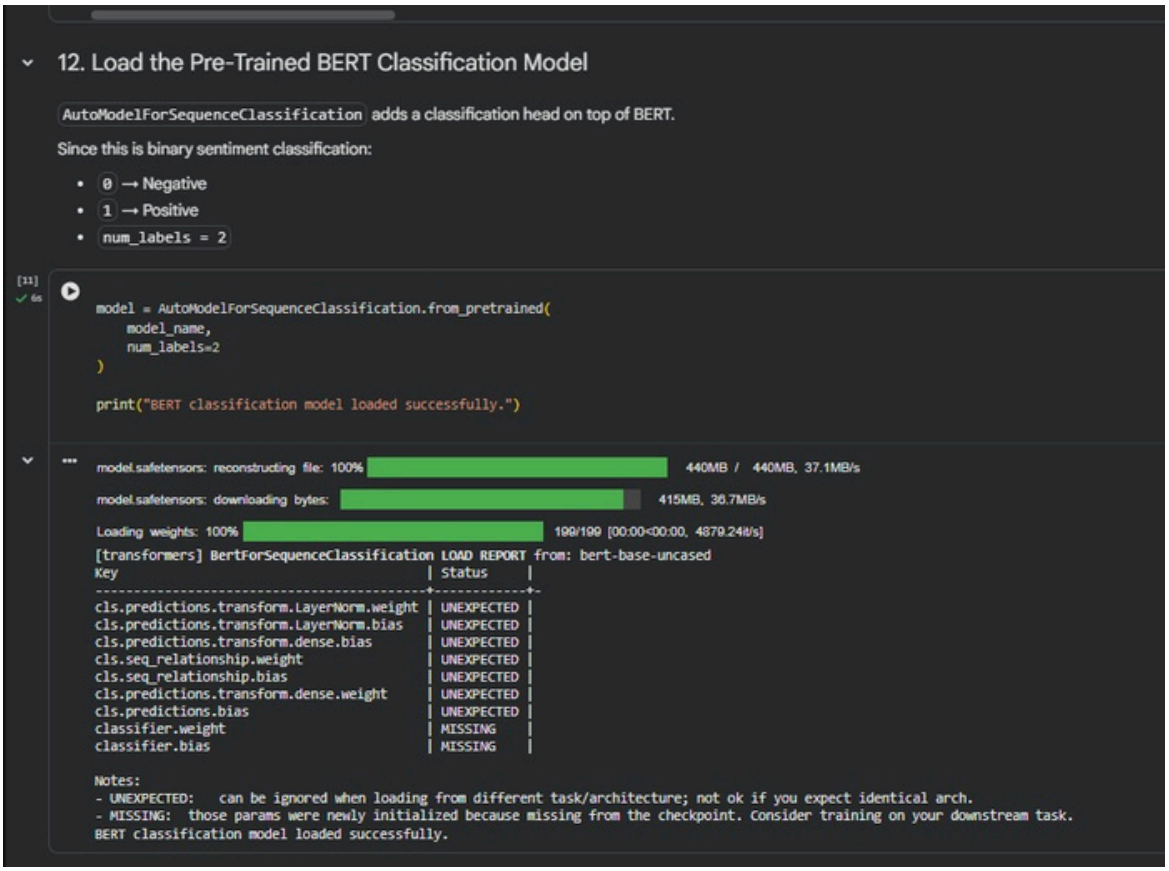



# *8. Load the Pre-Trained BERT Model*

Thebert-base-uncased model isalreadypre-trained ongeneral language data. For this task, a sequence classification head with two output classes is used.

```
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
```

num_labels=2
)




### *8.1 Move Model to GPU or CPU*

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
print("Using device:", device)
```

## *9. Model Training and Fine-Tuning*

Fine-tuning adaptsthe pre-trained BERT model tothespecific sentiment classification task. The model parameters are updated using the labeled IMDB reviews.

### *9.1 Evaluation Metric*

```
accuracy = evaluate.load("accuracy")

defcompute_metrics(eval_pred):
predictions, labels = eval_pred
predictions = np.argmax(predictions, axis=1)
return accuracy.compute(
    predictions=predictions,
    references=labels
  )
```

## *9.2 Training Arguments*

```
training_args=TrainingArguments(
    output_dir="./bert_sentiment_results",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_strategy="steps",
    logging_steps=100,
    load_best_model_at_end=True,
    report_to="none"

)
```

## *9.3 Create Trainer*

```
trainer=Trainer(
    model=model,
    args=training_args,
    train_dataset=train_tokenized,
    eval_dataset=test_tokenized,
    processing_class=tokenizer,
    compute_metrics=compute_metrics
)
```

## *9.4 Fine-Tune BERT*

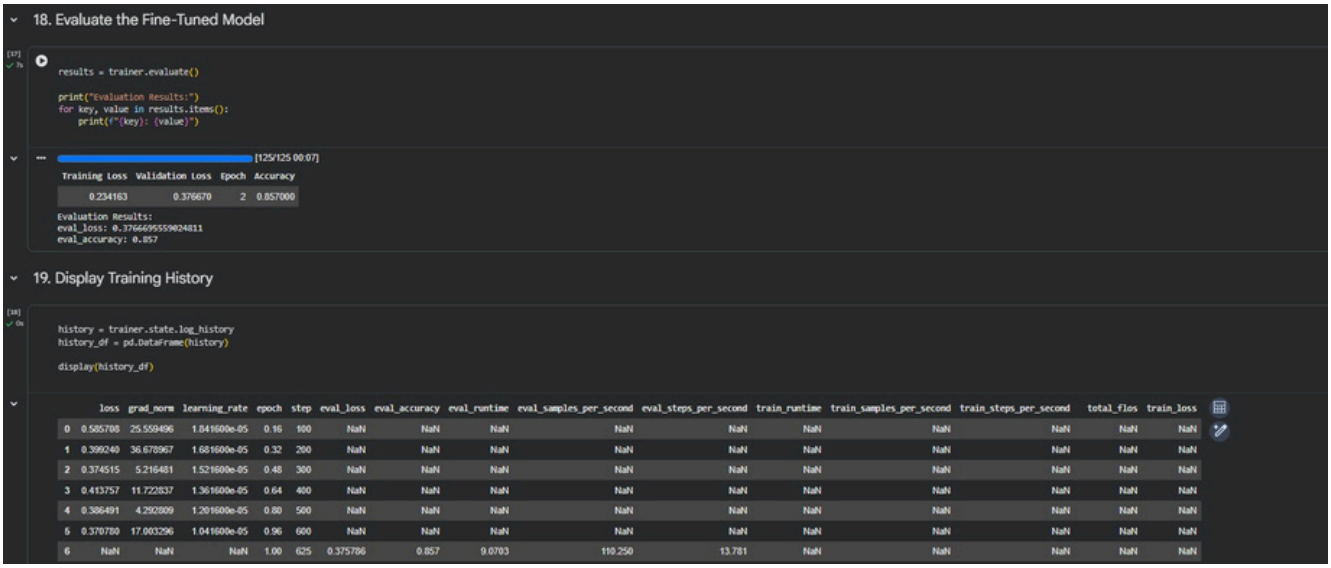
```
train_result = trainer.train()
```




# *10. Model Evaluation*

```
results = trainer.evaluate()

print("Evaluation Results:")
for key, value in results.items():
    print(f"{key}: {value}")
```



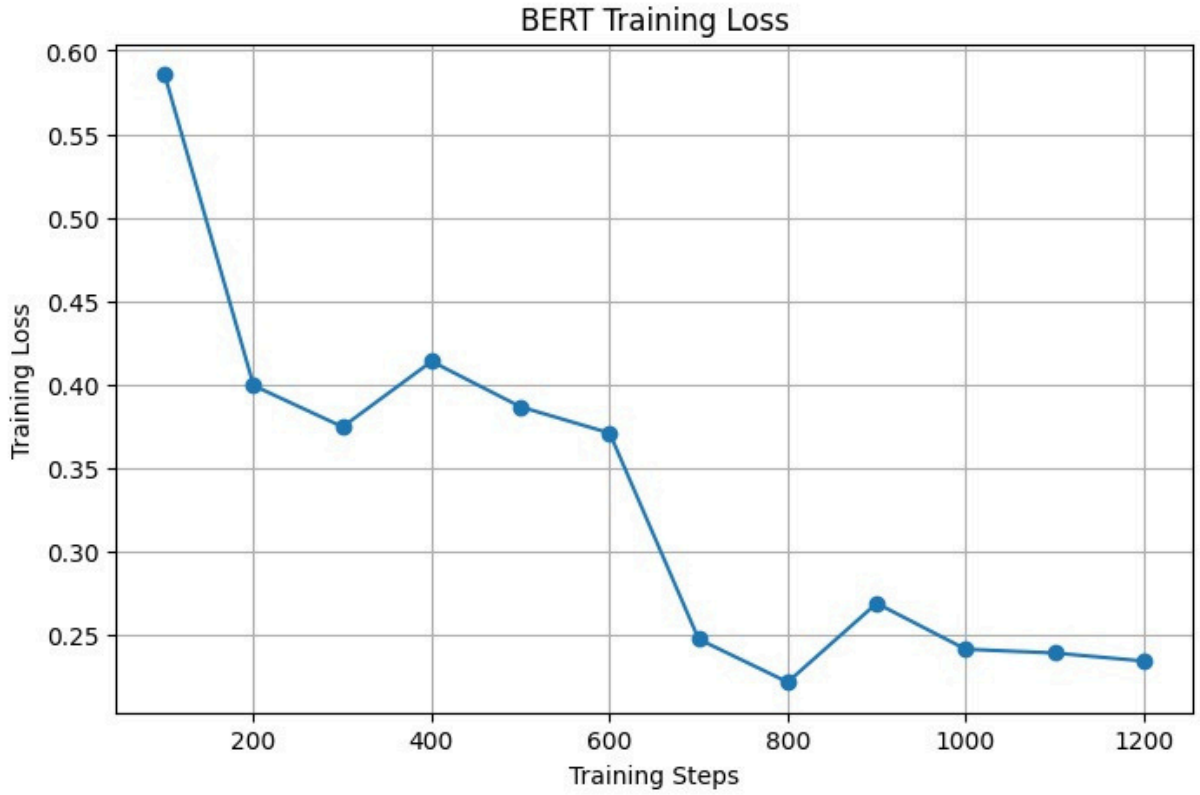

## *10.1 Training History*

```
history= trainer.state.log_history
history_df = pd.DataFrame(history)
display(history_df)
```

## *10.2 Training Loss Graph*

```
train_loss=history_df[history_df["loss"].notna()]

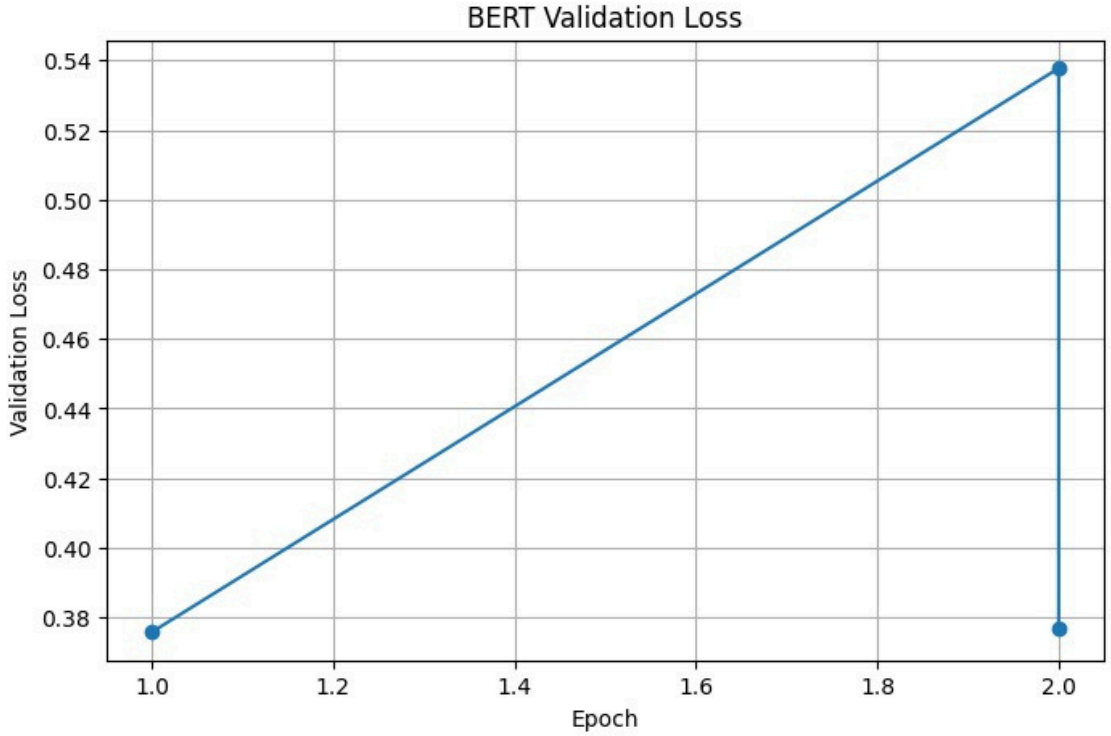
plt.figure(figsize=(8, 5))
plt.plot(train_loss["step"], train_loss["loss"], marker="o")
plt.xlabel("Training Steps")
plt.ylabel("Training Loss")
plt.title("BERT Training Loss")
plt.grid(True)
plt.show()
```




## *10.3 Validation Loss Graph*

```
eval_loss=history_df[history_df["eval_loss"].notna()]

plt.figure(figsize=(8, 5))
plt.plot(eval_loss["epoch"], eval_loss["eval_loss"], marker="o")
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.title("BERT Validation Loss")
plt.grid(True)
plt.show()
```




## *11. Confusion Matrix*

```
prediction_output = trainer.predict(test_tokenized)

predicted_labels = np.argmax(
    prediction_output.predictions,
    axis=1
)

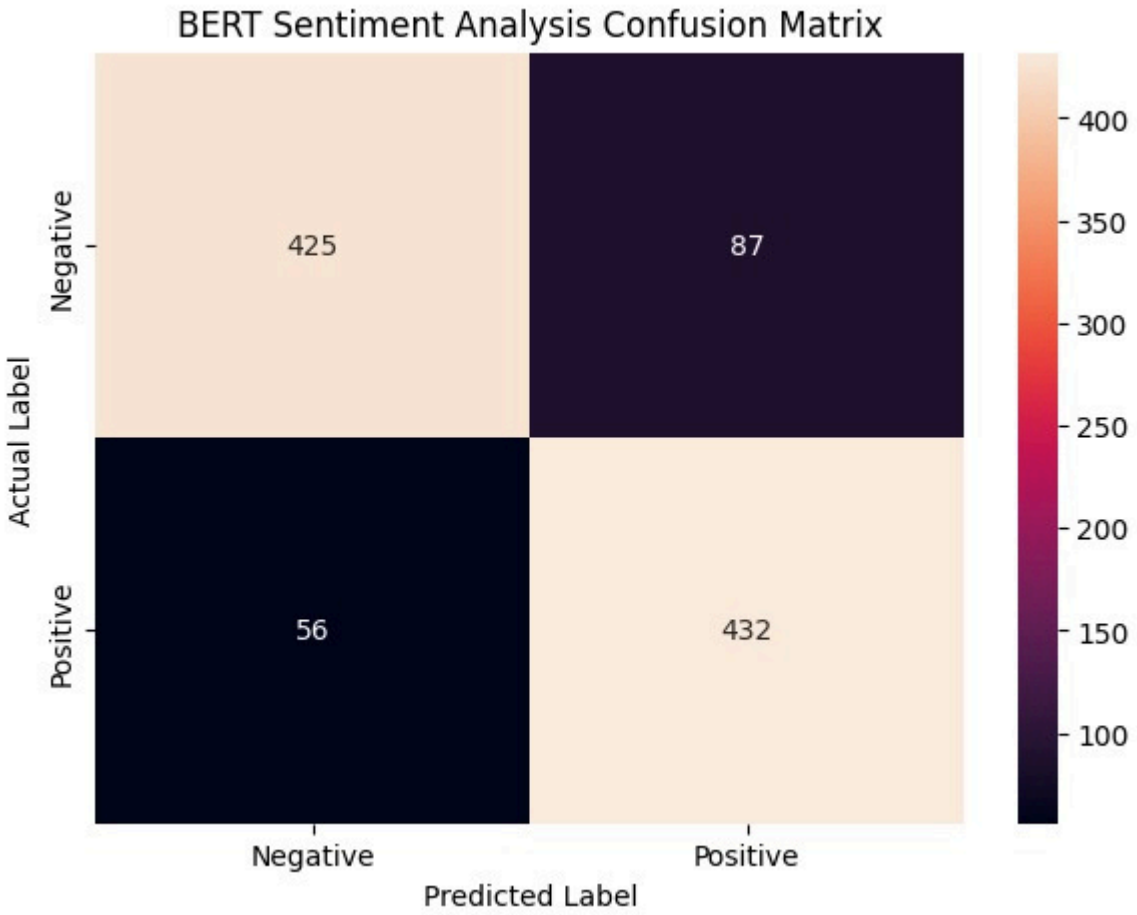
actual_labels = prediction_output.label_ids

cm = confusion_matrix(actual_labels, predicted_labels)
print(cm)

plt.figure(figsize=(7, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=["Negative", "Positive"],
    yticklabels=["Negative", "Positive"]
)

plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("BERT Sentiment Analysis Confusion Matrix")
plt.show()
```




## *12. Classification Report*

```
report = classification_report(
    actual_labels,
    predicted_labels,
    target_names=["Negative", "Positive"]
)

print(report)
```



# *13. Prediction on Custom Reviews*

Thetrainedmodelcanalsobeusedtoclassifynewreviews that were not present in the training dataset.

```
custom_reviews = [
    "This movie was absolutely fantastic. I really enjoyed it.",
    "The movie was boring and a complete waste of time.",
    "The acting was excellent and the story was very interesting.",
    "I hated this movie. It was terrible.",
    "It was an average movie, nothing special."
]

inputs = tokenizer(
    custom_reviews,
    padding=True,
    truncation=True,
    max_length=128,
    return_tensors="pt"
)

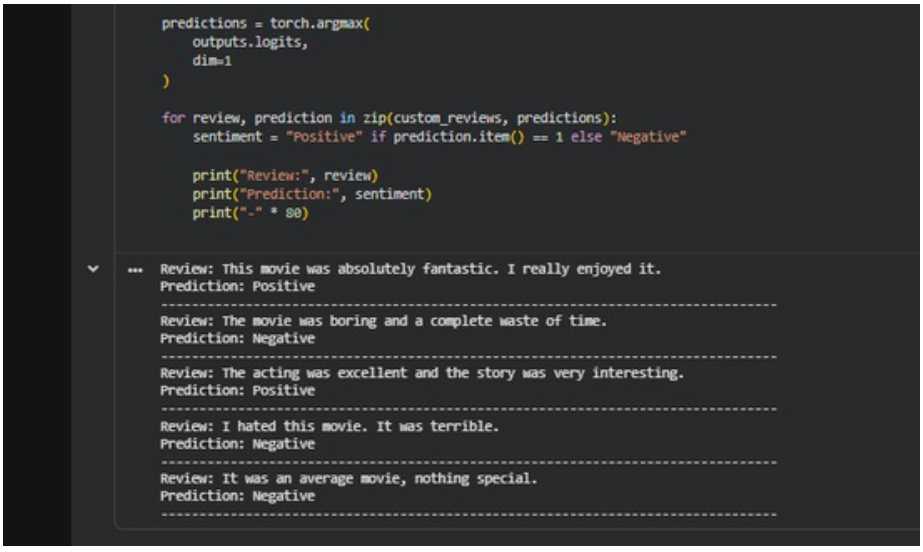
inputs = {key: value.to(model.device) for key, value in inputs.items()}

model.eval()

with torch.no_grad():
    outputs = model(**inputs)

predictions = torch.argmax(outputs.logits, dim=1)

for review, prediction in zip(custom_reviews, predictions):
    sentiment = "Positive" if prediction.item() == 1 else "Negative"
    print("Review:", review)
    print("Prediction:", sentiment)
    print("-" * 80)
```

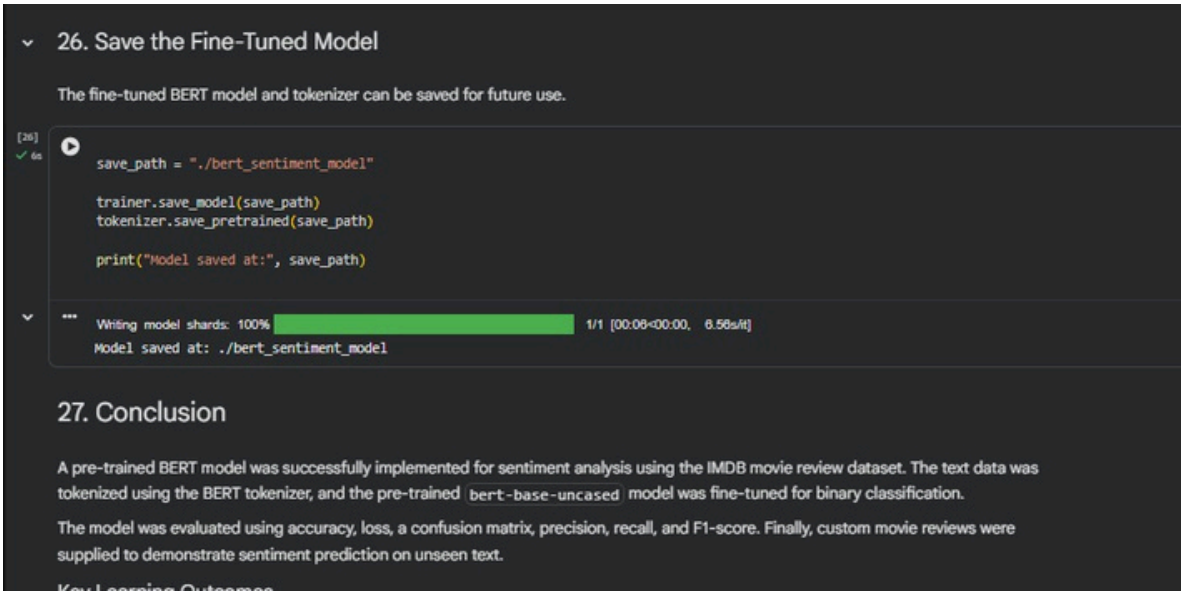


## *14. Save the Fine-Tuned Model*

```
save_path = "./bert_sentiment_model"

trainer.save_model(save_path)
tokenizer.save_pretrained(save_path)

print("Model saved at:", save_path)
```




## *15. Results and Discussion*

Thepre-trained BERTmodelwassuccessfully fine-tuned for binary sentiment analysis using the IMDB movie review dataset. The model performance is evaluated using accuracy, precision, recall, F1-score and a confusion matrix.

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| Negative | 0.88 | 0.83 | 0.86 | 512 |
| Positive | 0.83 | 0.89 | 0.86 | 488 |
| accuracy | | | 0.86 | 1000 |
| macro avg | 0.86 | 0.86 | 0.86 | 1000 |
| weighted avg | 0.86 | 0.86 | 0.86 | 1000 |

## *16. Conclusion*

Apre-trainedBERTmodel was implemented successfully for sentiment analysis. The IMDB dataset was tokenized using a BERT tokenizer, and the bert-base-uncased model was fine-tuned for binary classification. The trained model was evaluated using multiple classification metrics and a confusion matrix. Custom movie reviews were also classified to demonstrate the practicaluse of the fine-tuned model.